

Mable Audience Builder

Software Engineering Intern Assignment

Context

Mable helps teams turn product data into useful decisions. One common workflow is defining an audience from anonymous customer behavior, then understanding why a person belongs to that audience.

Build a small **Mable Audience Builder**. It has two independently runnable applications:

- A backend that evaluates audience rules against synthetic event data.
- A frontend where an operator builds a rule and previews the resulting audience.

The frontend must consume the backend API. We are assessing a clean, defensible implementation, not visual spectacle or the number of libraries used.

Timebox and Submission

- Spend no more than **4 hours** on the required scope.
- Submit within **72 hours** of receiving the task unless another deadline is agreed in writing.
- Use a public Git repository or provide one we can access.
- Use TypeScript for the frontend. The backend may use TypeScript or Go.
- AI coding tools are allowed. You remain responsible for the submitted code.

There is no deployment requirement. A reviewer must be able to run the project locally from the README.

Product Scenario

An operator wants to create an audience such as:

Users who viewed a product at least twice in the previous 7 days but did not purchase in that period.

All data must be synthetic and anonymous. Use these event types:

- `page_view`
- `product_view`
- `add_to_cart`
- `checkout_started`
- `purchase`

The assignment does not require login, external integrations, a drag-and-drop rule builder, event ingestion, or deployment.

Repository Layout

Keep both applications independently runnable:

```
frontend/  
backend/  
docs/  
  DESIGN.md  
  AI_USAGE.md  
README.md
```

Backend Requirements

Audience Preview API

Implement POST /v1/audiences/preview.

The request must accept:

```
{
  "name": "Viewed but not purchased",
  "asOf": "2026-09-29T00:00:00.000Z",
  "conditions": [
    {
      "eventType": "product_view",
      "operator": "at_least",
      "count": 2,
      "withinDays": 7
    },
    {
      "eventType": "purchase",
      "operator": "exactly",
      "count": 0,
      "withinDays": 7
    }
  ]
}
```

Required behavior:

- All conditions are combined with **AND**.
- Support the operators `at_least` and `exactly`.
- `asOf` makes the preview reproducible. The API must evaluate events relative to it, rather than relying on the server's current clock.
- Validate the request and return a consistent JSON error response for invalid rules.
- Return the audience size and the matched anonymous users. Include enough per-user evidence for an operator to understand why each user matched.
- Do not store or return personal data, credentials, or payment details.

Suggested response shape:

```
{
  "name": "Viewed but not purchased",
  "asOf": "2026-09-29T00:00:00.000Z",
  "total": 2,
  "members": [
    {
      "anonymousId": "anon_123",
      "evidence": [
        { "eventType": "product_view", "observedCount": 3 },
        { "eventType": "purchase", "observedCount": 0 }
      ]
    }
  ]
}
```

Also implement GET /health with a lightweight JSON response.

Backend Expectations

- Keep event data in a database, not only an in-memory array. SQLite is sufficient; PostgreSQL is also acceptable. Seed positive matches, non-matches, and boundary cases.
- Validate input at the HTTP boundary and use safe database-query mechanisms.
- Write meaningful automated tests for the audience evaluator and error behavior.
- Add enough request/error logging to make local failures diagnosable without exposing event payloads unnecessarily.

Frontend Requirements

Build one operator-facing screen with:

- 1 **Audience definition:** add and remove conditions; select event type, operator, event count, and time window.
- 2 **Preview action:** submit the definition to the backend. The frontend must not calculate audience membership itself.
- 3 **Results:** show the audience size and a clear list or table of matching anonymous users with their matching evidence.
- 4 **Product-quality states:** loading, empty, validation, and API-error states should be understandable. An API failure needs a visible retry path.

The frontend must use a configurable backend base URL, accessible semantic controls, and keyboard-operable rule editing. Keep request code, server data, and local form state reasonably separate so another engineer can follow the flow.

Engineering Expectations

- README.md with prerequisites, exact run/test commands, and a short explanation of how to preview an audience.
- docs/DESIGN.md (maximum 750 words) describing your data model, rule-evaluation choice, time-window decision, and one realistic scaling or product trade-off.
- docs/AI_USAGE.md listing any AI tools used and the role they played.

Make at least three meaningful commits. Do not commit secrets, real customer data, or generated build artifacts. Keep dependencies modest and explain any substantial dependency.

Review

We will run the project, inspect the code and tests, and discuss your decisions in a follow-up conversation. Be prepared to walk through the implementation and make a small change to the submitted code.

We assess end-to-end correctness, backend data reasoning, frontend state handling, readability, test quality, scope control, and ownership of the submitted code.